

Class 05 — Student Notes

Bong Study Hub — Foundation Batch 2026

Your First Python Program

From English Instructions → Python Code

These notes summarize what we covered in class. Use them to revise, not to learn the topic for the first time — the real learning happened in the room, at the keyboard, through the "About Me" program you wrote and ran yourself. Only `print()`, comments, and reading simple errors appear here — variables, `input()`, and everything else come in later classes.

The Big Question

A computer only ever does exactly what it's told — nothing more, nothing less. It never fills in a gap the way a person listening to you would.

"If a computer only does exactly what it's told, how do we tell it something — in a way it can actually follow?"

We won't answer this with a rule to memorize. We'll answer it by actually writing and running code.

Quick connection to Class 01

Back in Class 01, we built this chain and never opened one of its boxes:

PROBLEM → LOGIC → ALGORITHM → PROGRAM → RESULT

We talked about algorithms and programs for four whole classes without ever writing one. Today, PROGRAM stops being a word on a board.

```
ALGORITHM
  ↓
PYTHON CODE
  ↓
  RUN
  ↓
OUTPUT / RESULT
```

By the end of today, you will have personally built and run this chain.

1. What Is a Programming Language?

English is full of gaps a listener fills in automatically. "Go that way and turn" works fine between two people — it would mean nothing to a computer, which never guesses what you meant.

PROGRAMMING LANGUAGE = a language with strict, precise rules — a computer can follow it with zero guessing.

Programming languages are still languages people read and write. They're just far less forgiving than English about *exactly* how you write something.

2. Meet Python

Python is one programming language among many. It was deliberately designed to read almost like plain instructions, which is exactly why we're starting here — the underlying ideas of programming

show through more clearly in Python than in most other languages.

Remember: Python isn't "the" programming language. It's one of many, each suited to different jobs. We're starting here because it gets out of the way and lets the ideas underneath show through.

3. Where Code Runs

Two areas matter today:

- The **editor** — where you type your instructions.
- The **output** — where you see what happened after you run them.

That's genuinely the whole mental model for this class.

4. Your First Line: `print()`

```
print("Hello, World!")
```

- `print` tells Python: **display something**.
- The parentheses hold **what** to display.
- The quotes mark the text as literal text — more on this in Section 6.

Run this line, and `Hello, World!` appears in the output. That's a real, complete program — small, but not a toy. Every program you'll ever write does this same basic thing: instructions in, output out.

Remember Class 01? INPUT → PROCESS → OUTPUT. You just watched it happen for real. Your code was the process. The text on screen is the output.

5. Running the Program

"Running" a program means telling the computer to actually carry out its instructions, one at a time, and show you what happens.

```
print("Hello, my name is ____!")
```

Fill in your own name and run it — the output is yours because you wrote it.

6. Strings Need Quotes

Watch what happens without quotes:

```
print(Hello, World!)
```

This causes an **error** — Python no longer knows that `Hello, World!` is just text to display. Without quotes, it tries to treat those words as more instructions and gets confused.

STRING = text data, written between matching quotes. Quotes mark exactly where the text starts and where it ends.

Single quotes (`'...'`) and double quotes (`"..."`) both work — just make sure the one you open with matches the one you close with.

7. Multiple Instructions, In Order

```
print("Hello, my name is Priya.")  
print("I am learning Python.")  
print("This is my very first program!")
```

Python runs these **top to bottom, exactly as written** — no reordering, no guessing a "smarter" order.

This is exactly what "algorithm" meant in Class 01 — a sequence of steps, followed in order. Swap two lines, and the output swaps too.

8. Comments

```
# This program introduces me  
print("Hello, my name is Priya.") # prints my name  
print("I am learning Python.")
```

Anything after a `#` on a line is a **comment**. Python skips it completely — it's not an instruction at all, and running the program above produces exactly the same output as if the comments weren't there.

Comments are for people, not computers. They explain what code does to whoever reads it next — a teammate, or you, months from now.

9. When Syntax Breaks: Errors

Two small examples, broken on purpose:

A missing closing quote:

```
print("Hello, World!)
```

→ roughly: `SyntaxError: unterminated string literal` — Python is saying a piece of text that started with a quote never found its matching closing quote.

Wrong capitalization:

```
Print("Hello, World!")
```

→ roughly: `NameError: name 'Print' is not defined` — Python is case-sensitive; `print` and `Print` are two completely different words to it.

An error is not a judgment on you. It's the most useful thing Python can do for you — tell you exactly where it got confused, instead of failing silently. Every programmer sees hundreds of these. Read the last line first — it usually names the actual problem in plain words.

10. The Write–Run–Read–Fix Loop

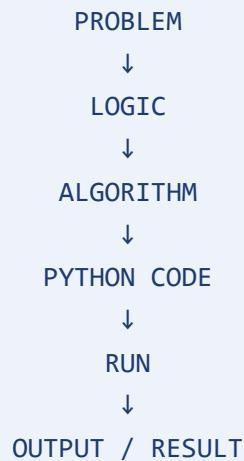
```
WRITE → RUN → READ → FIX → (RUN again)
```

This is what programming actually feels like, day to day. Write some instructions. Run them. Read what happened — output, or an error. Fix whatever needs fixing. Run again.

Remember: nobody writes a perfect program on the first try — not beginners, not experienced programmers. This loop *is* the normal way it works.

10.5 Connecting It All

Let's put the whole day on one chain — starting all the way back at Class 01:



The top three lines are exactly Class 01's chain. Today opened the box that used to just say **PROGRAM** — it's **PYTHON CODE**, then **RUN**, then the **RESULT** you actually watched appear on screen. Nothing about Class 01 changed. We just finally did it.

Try It Yourself — My First Program

Write a short program called "**About Me.**" It needs:

- At least **three** `print()` lines.
- At least **one** comment (`#`).

For example:

```
# About Me
print("Hello, my name is ____.")
print("I am learning Python for the first time.")
print("My favorite subject is ____.")
```

Fill in your own details, run it, and then try breaking it on purpose — remove a quote, or misspell `print`. Read the error. Then put it back and run it again. That's the write-run-read-fix loop, start to finish, on code that's entirely yours.

Quick Recap

- A computer needs **precise, unambiguous instructions** — it never fills in gaps the way a person listening to you would.
- **Python** is a programming language: one of many, chosen here because it's readable.
- `print(...)` **displays** whatever text is inside the parentheses.
- Text needs **quotes** so Python knows exactly where it starts and ends.
- Instructions run **top to bottom, in order** — exactly as written.
- A **comment** (`#`) is ignored completely by Python — it's for people.

- An **error** is normal, useful feedback, not a sign you broke anything — read the last line first.
- Programming is a loop: **write** → **run** → **read** → **fix** → **run again**.
- Class 05's chain connects directly onto Class 01's: PROBLEM → LOGIC → ALGORITHM → PYTHON CODE → RUN → OUTPUT/RESULT.

Key Terms

Term	Meaning
Program	A precise sequence of instructions a computer can carry out.
Python	A programming language, designed to be readable — one of many that exist.
Code / Source code	The actual text of the instructions we write.
<code>print()</code>	A built-in instruction that displays text as output.
String	Text data, written between matching quotes.
Comment (#)	A line, or part of a line, that Python completely ignores.
Run / Execute	Telling the computer to carry out the program's instructions.
Syntax	The precise rules for how Python code must be written.
Syntax error	What Python reports when code breaks those rules.
Output	What the program displays as a result of running.

Think About It

Try answering these in your own words — you don't need exact wording, just the right idea.

1. Why can't we just type plain English at a computer?
2. What does `print(...)` do?
3. Why does the text inside `print()` need quotes?
4. If you write four `print()` lines, in what order will they run?
5. What does Python do with a comment — and who are comments actually for?
6. Is getting an error a sign you did something badly wrong? Why or why not?

Final Takeaway

"You didn't just learn about programming today — you did it. Every program you will ever write, no matter how large, is built from exactly this: precise instructions, run in order, read carefully when they go wrong."

Right now, every piece of text in your programs is fixed — typed once and never changing. What if a program needed to remember something, or work with a value that changes? That's exactly where we pick up next class.